

结构的根基

基础数据结构 · 8题 · C++ & Python 双语对照

NQ122 **AcWing 826** AcWing 826

AcWing 826

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>

using namespace std;

const int N = 100010;

// head 表示头节点的下标, e[N]表示节点i的值
// ne[N]表示节点i的next指针是多少, idx存储当前已经用
// 到了哪个节点
int head, e[N], ne[N], idx;

void init()
{
    head = -1; // 一开始设置为-1
    idx = 0; // 当前用到了0个节点
}

// 将x插入到头节点
void add_to_head(int x)
{
    e[idx] = x, ne[idx] = head, head = idx, idx++;
}

// 将x插到下标是k的点后面
void add(int k, int x)
{
    e[idx] = x, ne[idx] = ne[k], ne[k] = idx, idx++;
}

// 将下标是k的点后面的点删掉
void remove(int k)
{
    ne[k] = ne[ne[k]];
    // idx--;
}

int main()
{
    int m; cin >> m;

    init(); // 初始化

    while (m --)
    {
        int k, x;
        char op; // 操作字符

        cin >> op;
        if (op == 'H')
        {
            cin >> x;
            add_to_head(x);
        }
        else if (op == 'D')
        {
            cin >> k;
            if (!k) head = ne[head]; // 删除头节点
            else remove(k-1);
        }
        else
        {
            cin >> k >> x;
            add(k-1, x);
        }
    }

    // 打印单链表

```

Python

AcWing 826

```
for (int i = head; i!=-1; i = ne[i]) cout <<
e[i] << ' ';
cout << endl;
}
```

NQ123 AcWing 828 AcWing 828

AcWing 828

输入 见原题

输出 见原题

范围 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```
#include <iostream>

using namespace std;
const int N = 100010;

int m;
int stk[N], tt; //数组模拟栈

int main()
{
    cin >> m;
    while (m --)
    {
        string op;
        int x;

        cin >> op;
        if (op == "push") //栈顶指针+1, 把数压入数组
        {
            cin >> x;
            stk[++tt] = x; //压入一个元素
        }
        else if (op == "pop") tt --; //修改栈顶指针即可
        else if (op == "empty") cout << (tt? "N
0":"YES") << endl; //tt>0表示非空
        else cout << stk[tt] << endl; //返回栈顶
    }
}
```

Python

```
# AcWing 828
```

NQ124 AcWing 829 AcWing 829

AcWing 829

输入

 见原题

输出

 见原题

范围

 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```
#include <iostream>
using namespace std;
const int N = 1000007;
int q[N], hh, tt=-1;

int main(){
    int m;
    cin>>m;
    while (m--){
        int x;

        string op;
        cin>>op;

        if (op == "push") {
            //(1) "push x" - 向队尾插入一个数x;
            cin>>x;
            q[++tt] = x;
        } else if (op == "pop")// (2) "pop" - 从队头
            弹出一个数;
            hh++;
        else if (op == "empty") // (3) "empty" -
            判断队列是否为空;
            cout<<(hh<=tt?"NO":"YES")<<endl;
        else cout<<q[tt]<<endl;// (4) "query" -
            查询队头元素。
        }

        return 0;
    }
}
```

Python

AcWing 829

NQ125 AcWing 3302 AcWing 3302

AcWing 3302

- 输入

见原题
- 输出

见原题
- 范围

见原题

输入	输出
见原题	见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <stack>
#include <unordered_map>

using namespace std;

stack<int> num; //存储计算过程中的数字
stack<char> op; //存储操作符

void eval()
{
    auto b = num.top(); num.pop(); //第二个数
    auto a = num.top(); num.pop(); //第一个数
    auto c = op.top(); op.pop();

    int x;
    if (c == '+') x = a + b;
    else if (c == '-') x = a - b;
    else if (c == '*') x = a * b;
    else x = a / b;
    num.push(x);
}

int main()
{
    //优先级, 可扩展为其他符号的优先级
    unordered_map<char, int> pr{{'+', 1}, {'-', 1},
    {'*', 2}, {'/', 2}};
    string str; //输入都存储在字符串内
    cin >> str;
    for (int i = 0; i < str.size(); i++)
    {
        auto c = str[i]; //读入每个char
        if (isdigit(c)) //判断是否是数字
        {
            int x = 0, j = i;
            while (j < str.size() && isdigit(str[j]))
                x = x * 10 + str[j++] - '0';
            i = j - 1; //调整i的位置
            num.push(x); //压栈
        }
        else if (c == '(') op.push(c);
        else if (c == ')')
        {
            //把前面所有的表达式都出栈运算
            while (op.top() != '(') eval();
            op.pop();
        }
        else
        {
            //根据操作符的优先级执行运算
            while (op.size() && op.top() != '(' && pr
[op.top()] >= pr[c])
                eval();
            op.push(c);
        }
    }
    //处理剩余的操作符
    while(op.size()) eval();
    cout<<num.top() <<endl;
    return 0;
}

```

Python

AcWing 3302

NQ126 AcWing 830 AcWing 830

AcWing 830

输入 见原题**输出** 见原题**范围** 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md**C++**

```
#include <iostream>
using namespace std;
const int N=100007;
int n;
int stk[N],tt;//数组模拟栈实现单调栈
int main ()
{
    cin>> n;
    for (int i=0; i< n; i++){
        int x; cin>>x;
        while (tt && stk[tt]>= x)//tt为空, 并且单调栈顶元素比x大
            tt--;//出栈, 这些数永远不会用到
        if (tt) cout<<stk[tt]<<' '; //栈顶就是比x小的元素, 直接输出
        else cout<<-1<<' '; //x左边没有任何数比它小

        //入栈
        stk[++tt] = x;
    }
    return 0;
}
```

Python

AcWing 830

NQ127 AcWing 154 AcWing 154

AcWing 154

输入 见原题**输出** 见原题**范围** 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
using namespace std;

const int N=10000007;

int n,k;
int a[N],q[N]; //数组模拟队列
int hh=0,tt=-1; //hh队头,tt对尾

void initQueue(){
    hh=0; tt=-1;
}

int main()
{
    scanf("%d%d",&n,&k);
    for (int i=0; i<n; i++) scanf("%d",&a[i]); //读入数字

    //求滑动窗口内的最小值
    initQueue(); //初始化队列,队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front删除队头
        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt && a[q[tt]]>= a[i]) tt--; //pop back删除队尾
        q[++tt] = i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最小值
    }
    puts("");

    //求滑动窗口内的最大值
    initQueue(); //初始化队列,队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front删除队头
        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt && a[q[tt]]<=a[i]) tt--; //pop back删除队尾
        q[++tt] = i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最大值
    }
    puts("");
}

```

Python

AcWing 154

NQ128 AcWing 831 AcWing 831

AcWing 831

输入

见原题

输出

见原题

范围

见原题

输入	输出
见原题	见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
using namespace std;
const int N = 100007, M = 1000007;
int n, m;          //n为待匹配字符串长度
char p[N], s[M];   //p为模板,s搜索范围
int ne[N];         // next数组,初始值为0

int main()
{
    // 第一行输入整数N, 表示字符串P的长度。
    // 第二行输入字符串P。
    // 第三行输入整数M, 表示字符串S的长度。
    // 第四行输入字符串S。
    cin >> n >> p + 1 >> m >> s + 1; //字符串从1
    开始存储, cin会在末尾加上\0

    // 求next的过程 next[1]=0;从第二个字符开始计算n
    ext
    for (int i = 2, j = 0; i <= n; i++) //预处理
    p串计算前缀后缀的值
    {
        //双指针操纵与匹配过程一致
        while (j && p[i] != p[j + 1])
            j = ne[j];
        if (p[i] == p[j + 1])
            j++;
        ne[i] = j; //找到i的回退点j
    }

    // KMP 匹配过程
    for (int si = 1, pj = 0; si <= m; si++) //s
    i,pj为字符串指针, 从1开始算
    {
        while (pj && s[si] != p[pj + 1]) // 用p
        字符串pj+1字符与s串的si比较 (提前看一个字符)
            pj = ne[pj];                //遇到
        不匹配, 回退pj指针
        if (s[si] == p[pj + 1])
            pj++; //pj进一位
        // p串匹配成功
        if (pj == n)
        { //抵达p串的最后一个字符
            cout << si - n << " "; //输出出现位置
            的下标
            pj = ne[pj];                //回退,预备下
            一次的查找
        }
    }
    return 0;
}

```

Python

AcWing 831

NQ129 AcWing 839 AcWing 839

AcWing 839

 见原题

 见原题

范围

见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <algorithm>
#include <string.h>

using namespace std;

const int N = 100007;

// hp是heap pointer的缩写, 表示堆数组中下标到第k个插入的映射
// ph是pointer heap的缩写, 表示第k个插入到堆数组中的下标的映射
// hp和ph数组是互为反函数的
int h[N], ph[N], hp[N], cnt;

void heap_swap(int a, int b)
{
    swap(ph[hp[a]], ph[hp[b]]); // 堆数组的下标
    swap(hp[a], hp[b]); // 下标与第几个插入元素的映射
    swap(h[a], h[b]); // 堆元素
}

void down(int u)
{
    int t = u;
    if (u*2 <= cnt && h[u*2] < h[t]) t = u*2; // u*2左儿子
    if (u*2+1 <= cnt && h[u*2+1] < h[t]) t = u*2+1; // u*2+1右儿子
    if (u != t)
    {
        heap_swap(u, t);
        down(t);
    }
}

void up(int u)
{
    while (u/2 && h[u] < h[u/2])
    {
        heap_swap(u, u/2);
        u >>= 1;
    }
}

int main()
{
    int n, m=0;
    scanf("%d", &n);
    while(n --)
    {
        char op[5];
        int k, x;
        scanf("%s", op);
        // 在堆最后一个元素插入元素
        if (!strcmp(op, "I"))
        {
            scanf("%d", &x);
            cnt ++;
            m++;
            ph[m] = cnt, hp[cnt] = m;
            h[cnt] = x;
            up(cnt); // 插入后执行up(cnt)
        }
    }
}

```

Python

AcWing 839

```
    }  
    else if (!strcmp(op, "PM")) printf("%d\n", h  
[1]); //堆顶是最小值  
    else if (!strcmp(op, "DM"))  
    {  
        heap_swap(1, cnt);  
        cnt --;  
        down(1); //删除最小值后执行down(1)  
    }  
    else if (!strcmp(op, "D"))  
    {  
        scanf("%d", &k); //删除第k个元素  
        k = ph[k]; //取第k个元素下标  
        heap_swap(k, cnt); //与最后一个元素交换  
        cnt--;  
        up(k);  
        down(k);  
    }  
    else {  
        scanf("%d%d", &k, &x); //修改第k个数  
        k = ph[k];  
        h[k] = x;  
        up(k);  
        down(k);  
    }  
}  
return 0;  
}
```